Knowledge-Based Systems 329 (2025) 114406

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

Automated vulnerability score prediction through lightweight generative AI

Seyedeh Leili Mirtaheri [a,*], Andrea Pugliese [a], Valerio Pascucci [b]

[a] Department of Informatics, Modeling, Electronics and System Engineering, University of Calabria, Arcavacata, Italy
[b] Scientific Computing and Imaging Institute, and School of Computing, University of Utah, Salt Lake City, UT, 84112, USA

ARTICLE INFO

Keywords:
Generative AI
Vulnerability scoring
LLM
BERT
Transformer

ABSTRACT

Given the constantly increasing number of newly published vulnerabilities, manually assessing their scores (e.g., under the Common Vulnerability Scoring System) has become unfeasible. Recently, learning-based systems have been proposed to automatically predict vulnerability scores. Such systems use vulnerability indexing databases to train deep learning algorithms. However, their practical applicability has important limitations, including a high dependency on the quality and diversity of training data, and high computational requirements. In addition, vulnerability descriptions often do not follow the standard templates and are not rich enough with respect to the expected features. In this paper, we propose a novel architecture that takes advantage of both generative artificial intelligence and lightweight deep learning techniques to provide an efficient and effective solution for automated vulnerability scoring. Data extracted from the National Vulnerability Dataset is fed into a large language model layer, whose output (i.e., an augmented dataset) is then used in a lightweight fine-tuned BERTsmall layer. We provide the results of an extensive experimental assessment of the effect of both each layer of the architecture and end-to-end performances. The results suggest that the combination of GPT3.5-Turbo and BERTsmall provides the most effective accuracy-time trade-off. We also compare the performance of the proposed architecture with other LLMs, BERT models, and cutting-edge approaches. The results show good improvements in prediction quality also when compared to a recent technique that incorporates data from 66 different sources, including the NVD.

1. Introduction

Software vulnerabilities pose a wide variety of risks [15,45]. Identifying and understanding the severity of vulnerabilities is crucial to establishing effective patching schedules [13]. This requires prioritizing vulnerabilities based on their risk levels to promptly reduce the potential for harm or damage [2,5].

The Common Vulnerability Scoring System (CVSS)[1] provides evaluation metrics and a severity rating to assess vulnerabilities, by categorizing different aspects. This assists in determining which vulnerabilities impose the need for earlier responses.

Vulnerability assessment is a well-studied subject in the literature. Manual assessments by experts is very time-consuming. In addition, as reported in Fig. 1, there is a continuous and significant increase in the number of reported vulnerabilities [11,12]. This tremendous increase in both quantity and severity of vulnerabilities poses increasingly difficult challenges to traditional assessment methods.

Machine learning (ML) techniques have been used to provide time-efficient predictions of vulnerability scores. Existing methods often utilize supervised deep learning (DL) techniques and train a model using existing vulnerability datasets [46]. The robustness of predictions is one of the practical challenges in such methods, since their performance is highly dependent on the quality of training data [18]. Several studies have contributed to improving performances by proposing more complex models or integrating several sources for training data [22,50]. However, applications constantly demand less computational load, also for compliance with sustainable energy consumption. Besides, although collecting and integrating multiple data sources can provide better results, it may be expensive and in some cases may result infeasible [12].

A recent analysis of the published vulnerability descriptions shows a notable increase of the number of descriptions that do not follow standard templates [14]. This deviation from templates degrades the performance and the robustness of DL-based methods – vulnerability descriptions should encompass essential elements that are crucial for recognizing different aspects.

Recent advancements in large language models (LLMs) have transformed various sectors, including cybersecurity [6]. LLMs have proved capable of comprehending and producing human language, showing potential in enhancing security protocols [7,8]. Harnessing the capabilities of LLMs can aid experts in efficiently identifying high-risk vulnerabilities

* Corresponding author.
E-mail addresses: Mirtaheri@dimes.unical.it (S.L. Mirtaheri), andrea.pugliese@unical.it (A. Pugliese), pascucci@acm.org (V. Pascucci).

[1] https://nvd.nist.gov/vuln-metrics/cvss

https://doi.org/10.1016/j.knosys.2025.114406
Received 3 September 2024; Received in revised form 14 August 2025; Accepted 1 September 2025
Available online 4 September 2025
0950-7051/© 2025 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

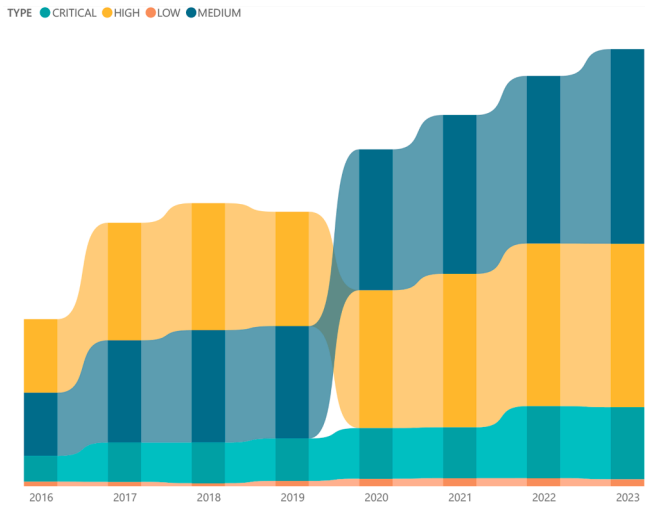

Fig. 1. Distribution of the vulnerability severity levels from 2016 to 2023.

[9,10]. Furthermore, incorporating LLMs into cybersecurity strategies can result in enhanced vulnerability assessment approaches and significant advancements for the industry, particularly in adapting to evolving threats [6].

The limitations of past work in addressing out-of-standard template vulnerability descriptions, together with the high computational requirements, motivated this work. Generally, there are two main challenges associated with software vulnerability scoring – enriching datasets and providing appropriate accuracy and precision. Several studies, including [12], aim to build extensive and high-quality datasets by crawling various sources like social media [3,4]. Despite the advances provided by such approaches, the gathered data results in huge datasets, with a high likelihood of noise, including unclean and/or repetitive entities with varying text formats, language, and styles. Such data does not necessarily guarantee substantial performance improvements, as it leads to more complex deep learning models, facing a more time-consuming training process to capture the nuances in the collected data. The experimental results obtained in [12] using template-based vulnerability descriptions based on datasets collected from various sources show that this approach is very effective in enhancing the accuracy performance of predictive models. Moreover, recent studies on vulnerability scoring models show that, despite their complex and large sizes, they still lack the required precision. The size of such models typically necessitates considerable training time, as well as computational resources. We explore the effect of template-based vulnerability descriptions augmented using generative AI. This enhances the quality of existing datasets in addition to facilitating predictive models by unifying the language and language patterns. To further improve performance, we propose a lightweight architecture based on a fine-tuned BERTsmall model – this speeds up the CVSS vector assessment process and holds a graceful balance between accuracy and computational load.

We use LLMs to generate augmented datasets based on CVE standard templates, using properly designed prompts. As in the case of the state-of-the-art [12,51], we use BERT transformer models to propose a lightweight architecture that is optimally fine-tuned to employ the augmented dataset. Our proposed architecture improves prediction without integrating new databases or introducing additional computational load. Extensive experiments support the intuition that by utilizing both LLMs and fine-tuned transformers, we are able to provide an efficient and effective automated vulnerability scoring system. To the best of our knowledge, we are the first to address the issue of out-of-template descriptions using LLMs.

The main contributions of this study can be summarized as follows:

- We study the effect of various LLMs, including GPT3.5-Turbo, Claude3 Haiku, and Claude1.2 Instant, to create augmented datasets based on the CVE official template. We feed the original public vulnerability descriptions to LLMs to enhance their context and clarity.[2] We show that GPT3.5-Turbo excels in generating comprehensive descriptions.
- We identify the LLM that provides the maximum accuracy in predicting CVSS vectors after evaluating several BERT models, including DistilBERT, BERTsmall, MediumBERT, and TinyBERT. We also compare the performance of BERT models based on common metrics.[3]
- We propose a lightweight architecture that employs BERTsmall for automated vulnerability score prediction. We fine-tune the model on the original data with a probabilistic learning approach. This enables us to handle uncertainties efficiently, thus expediting the generation of CVSS metrics. Instead of adding extensive layers to create a complex deep learning model, our proposed architecture provides a trade-off between accuracy and computational complexitload.

The remainder of this paper is organized as follows. In Section 2 we review related work on automated vulnerability score prediction. Section 3 describes the proposed architecture. The results of our experimental assessment are reported in Section 4. In Section 5 we discuss our findings and the limitations of our approach. Finally, Section 6 concludes the paper.

2. Related work

Automatic vulnerability scoring is a well-studied problem that can be addressed from different perspectives. In this section, we consider two main categories – ML-based and LLM-based approaches. In general, vulnerability descriptions have been a key element in the automated prediction of CVSS metrics [16,21,22]. The potential changes in descriptions can affect the performance of automated scoring methods [21]. Traditional methods used natural language processing (NLP) techniques to analyze these text-based descriptions. Kanakogi et al. study tf-idf (term frequency-inverse document frequency) methods and Doc2Vec [33]. They show the better performance of Doc2Vec over tf-idf in finding the most relevant attack pattern. In another work, Kuhn et al. [12] analyzed the NVD and the impact of collecting data from different sources. They proposed a deep learning model to predict and classify CVSS metrics. Guo et al. [35] proposed a neural-network-based approach to predict missing aspects. They trained their model with CVEs and showed the potential effectiveness of automated vulnerability assessment in aiding experts and reducing human efforts. Although NLP methods have been shown to be time-efficient compared with manual processing, they could not easily address other challenges such as out-of-template descriptions.

2.1. ML-based approaches

ML-based approaches typically aim to predict CVSS metrics and to identify public exploits using learned patterns. They have shown effectiveness with techniques such as convolutional neural network (CNN) classifiers, word embeddings, and long short-term memory (LSTM) models [14,19,20]. Linear support vector machines (SVM) have also been proposed to predict the vulnerability exploitation probability [23,24]. Likewise, random forests, regression, decision trees, light gradient boosting machines, and multi-layer perceptrons have been used [25–29].

Such works often focus on designing more efficient deep learning architectures or integrating diverse data sources. For instance, Chen et

[2] These augmented datasets are publicly available at https://github.com/Perfetto-AI-Research/Vulnerability-Datasets

[3] Software is publicly available at https://github.com/Perfetto-AI-Research/Automated-Vulnerability-Score-Prediction-through-Lightweight-Generative-AI/tree/main

al. [13] concentrated on data collected from Twitter, employing a graph convolutional network. Similarly, Khandpur et al. [39] and Zong et al. [40] used data from Twitter to study software vulnerabilities and potential attacks. The authors in [11,31,32] mainly focus on using deep learning and attention-based models to categorize CVSS metrics. Ognawala et al. [30] use random forests to predict user interaction metrics and the privileges required to exploit vulnerabilities. Logistic and linear regression methods are used in [11,34] to predict CVSS values based on NVD data. Jia et al. [17] also used artificial intelligence and proposed a framework for multi-step attacks, to enhance cybersecurity in smart city scenarios.

2.2. LLM-based approaches

Recent advancements in the development of LLMs have received significant attention in the field of cybersecurity. Zhang et al. [6] analyze various LLMs, exploring their potential applications in cybersecurity and highlighting efficient methods [36,37]. Some studies emphasize the application of GPT-based models through prompt engineering, enabling LLMs to be directed to the specific requirements of cybersecurity [38]. Incorporating LLMs, researchers often adapt models with either supervised fine-tuning or continual pre-training, i.e., a technique where pre-trained LLMs undergo further training using a large volume of unlabeled domain-specific data to enhance their applicability in cybersecurity [6,48].

Li et al. [41] focus on prioritizing vulnerabilities and highlighting the need for automated tools for assessing the severity and exploitability of vulnerabilities. They focus on using only descriptions and adopt a prompt-based learning method based on pre-trained language models (PLMs). Their experimental results show that prompt-based learning effectively utilizes PLM knowledge, outperforming traditional methods like fine-tuning PLMs and others, particularly in scenarios with limited data. Lu et al. [42] study the challenges of PLMs due to input format disparities and dataset sizes, which influence performance. They use prompt tuning to mitigate these challenges and assess various PLMs, prompt templates, and datasets. Their experiments confirm the effectiveness of prompt tuning on the accuracy of the models.

Malhotra and Vidushi [43] analyze data from vulnerability reports of five Mozilla software products and use text mining techniques to preprocess the data into feature vectors. They examine various vulnerability severity prediction models and machine learning algorithms, with a focus on medium and critical severity levels. In another study, Ren et al. [44] point out the challenges in software vulnerability identification and present a framework called ProRLearn. They adopt prompt tuning and reinforcement learning techniques – to address the limitations of fine-tuning PLMs in leveraging domain-specific knowledge, ProRLearn employs automatic prompt tuning to rephrase inputs effectively. In [12] the authors aim to improve the performance of existing methods by applying BERT models and integrating multiple data sources (66 datasets). Similarly, the authors in [49] explore the performance of BERT-based models in vulnerability description assessment, and have shown the effectiveness of such models in accurately identifying severe vulnerabilities.

2.3. Other generative AI approaches

Beyond LLMs, other generative AI (GenAI) frameworks are also rapidly proving their worth in the application of cybersecurity [57]. For example, a sequence-to-sequence generative model was applied to detect Trojan triggers in text samples to defend against backdoor attacks on deep neural network-based text classification systems, thereby increasing model robustness by identifying and mitigating such threats [57]. Within cybersecurity, Generative Adversarial Networks (GANs) are often put to use in data augmentation addressed to help solve imbalanced datasets or generate datasets in adversarial situations. GANs have been tested for vulnerabilities where attackers use them and learn hidden attributes concerning training data, thereby raising the privacy concerns of cybersecurity applications [58]. Beyond this, variants of GANs have been employed in model inversion attacks to reconstruct training data fairly faithfully, thus demonstrating some vulnerabilities of AI systems used in cybersecurity settings and hence calling for some protective measures [60].

Diffusion models are another advancement in GenAI, which often ends up being superior to GANs in output quality. The diffusion models are employed in cybersecurity for tasks like anomaly detection and robust learning. For example, a diffusion-based method in conjunction with local smoothing yields certifiably robust learning, which helps defend against adversarial perturbations to machine learning models for security tasks [59]. These generative models have applications in other domains as well, such as remote sensing; an efficient diffusion probabilistic model works towards the super-resolution of remote sensing images, speeding up the sampling process for reconstructing high-resolution images with a reduction in computation overhead [61]. Examples also include the improvement of low-quality images in digital forensics or the generation of synthetic vulnerability datasets and can be adapted to cybersecurity purposes. Inclusion of GANs and diffusion models in pipelines for vulnerability scoring, as explored with LLMs, can further contribute to dataset enrichment through simulation of diverse attack scenarios or augmentation of non-standard data, leading to improved predictive accuracy in resource-scarce settings.

Although the existing studies have many advantages, there are still challenges to be addressed and room for improvement. To the best of our knowledge, this is the first work that uses LLMs to address the problem of out-of-template descriptions. Additionally, as an appropriate trade-off between computational requirements and accuracy, we propose an architecture rather than just including additional layers into a deep learning model. We aim to provide a high accuracy such as the state-of-the-art [12], even when using a single source of data.

3. Proposed architecture

CVSS metrics are divided into three groups: Base, Temporal, and Environmental. Base metrics (Table 1) represent the intrinsic qualities of a vulnerability, which are constant over time and across environments. Temporal metrics reflect characteristics that may change over time but do not depend on the environment. Finally, environmental metrics are specific to environments. These metrics help adjust the Base and Temporal scores based on the environment's specific impact.

In this work, we employ BERT models including TinyBERT, BERTsmall, MediumBERT, DistilBERT, and BERT to predict scores [47]. We focus on BERT models for two reasons. First, because they have proved effective in NLP tasks and particularly in text categorization. Second, to appropriately compare with the state-of-the-art work in [12], which uses BERT models.

To further enhance the performance of BERT models, we fine-tune the BERT tokenization model, in order to make the pre-trained model tokenizer better handle domain-specific terms in vulnerability descriptions. We first develop a model comprising a one-classification layer alongside the pre-trained BERT tokenization model. The resulting model is then trained individually for each vulnerability metric, to better capture the nuances of the descriptions relating to each of these elements. The main motivation behind this choice is that each of these elements may relate to some keywords in the descriptions. The final personalized model is therefore more capable of representing these keywords, as the parameters of the BERT model are adjusted to optimize to the specific vulnerability element.

Variants of the BERT model are TinyBERT, BERTsmall, MediumBERT, and DistilBERT. TinyBERT is a compact version, designed for resource-limited environments, which achieves competitive performance with fewer parameters. BERTsmall aims at balancing complexity and performance and is more suitable for computational-efficient tasks.

Table 1
Summary of CVSS base metrics.

Metric	Values	Description
Exploitability Metrics		
Attack Vector (AV)	Network (N)	The vulnerability can be exploited remotely over a network.
	Adjacent Network (A)	The vulnerability can be exploited over a local network.
	Local (L)	The vulnerability requires local access to exploit.
	Physical (P)	The vulnerability requires physical access to exploit.
Attack Complexity (AC)	Low (L)	The attack is straightforward and does not require special conditions.
	High (H)	The attack requires specific conditions or complex setups.
Privileges Required (PR)	None (N)	No privileges are required to exploit the vulnerability.
	Low (L)	Low-level privileges are required to exploit the vulnerability.
	High (H)	High-level privileges are required to exploit the vulnerability.
User Interaction (UI)	None (N)	No user interaction is required.
	Required (R)	User interaction is required to exploit the vulnerability.
Impact Metrics		
Confidentiality (C)	None (N)	No impact on confidentiality.
	Low (L)	Limited impact on confidentiality.
	High (H)	Complete compromise of confidentiality.
Integrity (I)	None (N)	No impact on integrity.
	Low (L)	Limited impact on integrity.
	High (H)	Complete compromise of integrity.
Availability (A)	None (N)	No impact on availability.
	Low (L)	Limited impact on availability.
	High (H)	Complete compromise of availability.
Scope Metrics		
Scope (S)	Unchanged (U)	The impact is confined to the vulnerable component.
	Changed (C)	The impact extends beyond the vulnerable component.

MediumBERT is a mid-sized option between BERT and its more compact variants. Finally, DistilBERT is a smaller and more efficient variant with much faster training and almost the same performance [47].

We create augmented datasets based on CVE templates using multiple LLMs. GPT3.5-Turbo is well known for its powerful text generation performance, offering thorough and contextually appropriate descriptions. Claude3 Haiku is renowned for its efficiency and performance balance, providing a favorable trade-off between computational load and quality of the results. Claude1.2 Instant, given its emphasis on efficiency and speed, is well-suited for applications where prompt response times are essential.

We focus on the descriptions of vulnerabilities and consider the Base metrics (Table 1). We propose an end-to-end architecture that utilizes existing databases and a model that can be used in practical applications by providing a proper trade-off between accuracy and computational load. Unlike existing methods that mainly improve performance by utilizing a more complex and heavy deep learning model or by integrating several sources of data, we only employ one publicly available dataset. In the proposed architecture (Fig. 2) we try to address the following questions:

1. What is the impact of LLMs on generating vulnerability descriptions that adhere to the official CVE template?
2. Using the augmented CVE template dataset, which model can predict the CVSS vectors with the highest accuracy and minimum computational load?

Our architecture employs a two-phase approach. The first phase involves generating augmented datasets to enrich vulnerability descriptions and make sure the descriptions follow the official CVE template. We evaluate LLMs including GPT3.5-Turbo, Claude3 Haiku, and Claude1.2 Instant. These models are tasked with producing augmented datasets based on common vulnerabilities and exposures to CVE-specified templates.

In the second phase, we employ various pre-trained BERT models, including DistilBERT, BERTsmall, MediumBERT, TinyBERT, and BERT. These models are used to predict CVSS vectors based on the datasets generated in the first phase. We train and test BERT models on such datasets and look at performance metrics including accuracy, precision, recall, F1-score, and errors. In Fig. 2, the left-hand side component refers to the LLM-based vulnerability description generation, and the right-hand side includes the fine-tuned BERT model.

The vulnerability descriptions from the NVD are fed to the LLM using a specifically designed prompt, aimed to regenerating descriptions based on the official CVE template. These descriptions are then used to train and evaluate a fine-tuned, pre-trained BERT model to identify the value for each vulnerability metric, individually.

To determine the LLM with the best effectiveness, we examined the data generated by each of the tested models, using an array of BERT prediction models, to identify the impact of each set of produced descriptions, and determine which BERT model produces the most accurate predictions.

4. Experimental evaluation

We assessed the performance of the proposed architecture through extensive experiments in different scenarios. We measured the resulting quality and computational performance of BERT, DistilBERT, BERTsmall, TinyBERT, and BERT Medium on the original and augmented datasets generated by GPT3.5-Turbo, Claude1.2 (C1) Instant, and Claude3 (C3) Haiku. We evaluated quality using accuracy (A), precision (P), recall (R), F1-score (F), and Cohen's Kappa (C). The latter measures agreement between two classifications, and values close to 1 indicate perfect agreement. We also looked at mean absolute error (MAE) and mean squared error (MSE) to evaluate the performance in terms of final prediction errors. In addition, we compare with the state-of-the-art [12].

The experiments were carried out on a 64-bit system with 8 NVIDIA TU102 (GeForce RTX 2080 Ti) GPUs. The models were trained for 20 epochs, with a batch size of 16, 80 % train, and 20 % test data.

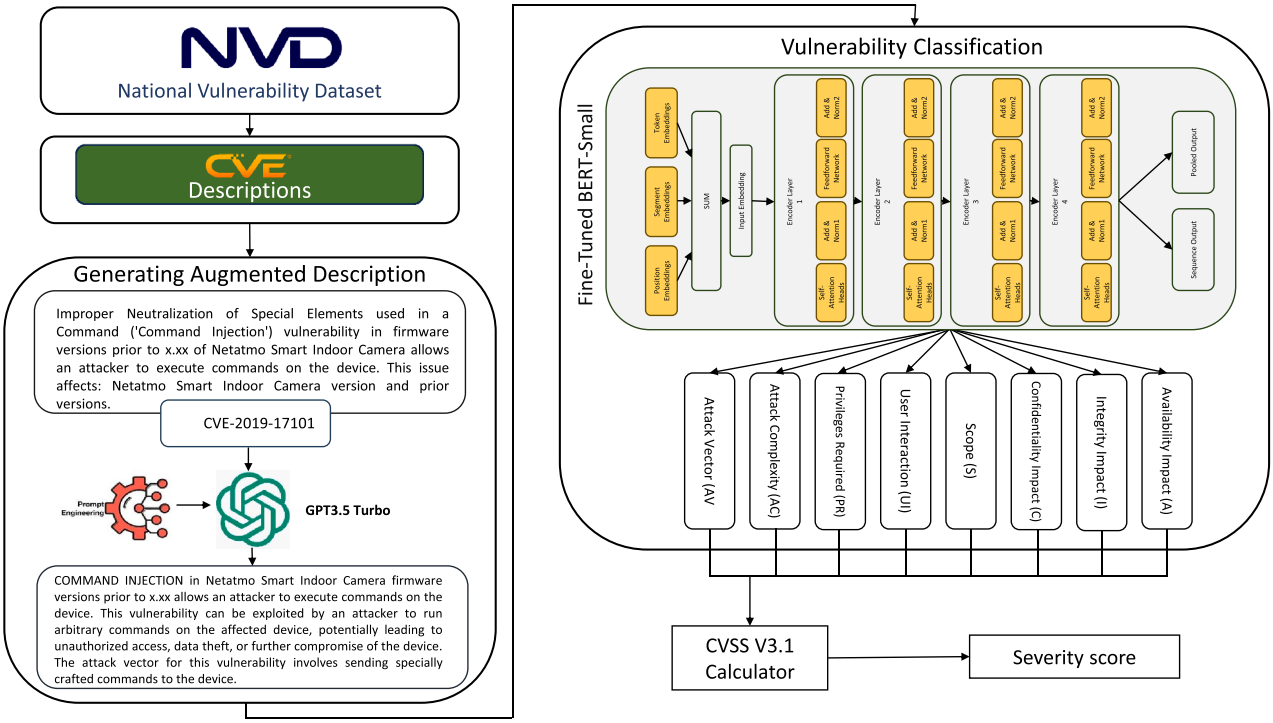

Fig. 2. An overview of the proposed architecture.

4.1. Datasets

The original dataset consists of 137,546 NVD vulnerability descriptions, along with the value for different metrics, and the official severity score. Using LLMs on the original dataset, we created three additional augmented datasets. We randomly selected 6370 descriptions, making sure that the ratio of labels in the extracted dataset is in accordance with the original data. This ensures essential attributes like label distributions are preserved accurately — the refined dataset mirrors the original characteristics for training and evaluation. This dataset is then utilized to generate the template-adhering descriptions using the LLMs. We used the following prompt:

> The CVE description for $\{cve_id\}$ is as follows:
> $\{description\}$
>
> The [DESCRIPTION] field should include an explanation of the attack type, the impact of the vulnerability, the software components affected, and any attack vectors.
>
> Descriptions often follow this template:
> "PROBLEM TYPE" in "AFFECTED COMPONENTS" causes "IMPACT" when "ATTACK VECTOR"
>
> Please generate a comprehensive and formatted description for this CVE based on the mentioned template and description.

This research used the standardized CVE template based on the MITRE CVE format: *[PROBLEM TYPE] in [PRODUCT/VERSION] causes [IMPACT] when [ATTACK]* because it captures the required attributes of critical vulnerabilities in much shorter but structured sentences, such as the kind of attack, the affected component, impact, and attack vectors. This pattern ensures that critical information is uniformly presented with minimal variability in definitions; thus, it will improve the performance of the model by standardizing the input data quality. The template parts were designed to be considered with CVSS Base metrics, allowing for direct mapping with factors that can weigh both exploitability and impact, which are central to accurate severity scoring.

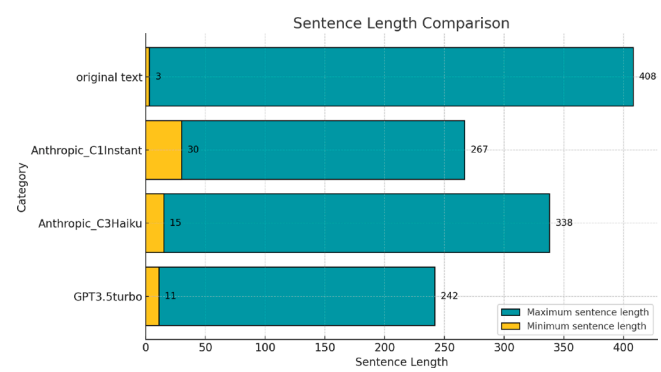

Fig. 3. Length of sentences in LLM-generated datasets.

Following this format, augmented descriptions could maintain clarity and relevance without generating noise but could improve their robustness for future downstream predictive models.

Fig. 3 shows the maximum and minimum length of sentences in the original and LLM-generated datasets.

We designed the prompt based on the official MITRE CVE description template [1]. As per the guidelines, each CVE description must adhere to the format *[PROBLEM TYPE] in [PRODUCT/VERSION] causes [IMPACT] when [ATTACK]*. This structure will describe the type of problem, the product/version that is affected, the impact caused by the problem, and under what circumstances the vulnerability can be exploited. Basically, according to MITRE, the description must contain a plain language explanation of the attack type, the impact the vulnerability could cause, what software components are affected, and relevant attack vectors, ensuring that each entry is unique and understandable. By aligning our prompt with the above well-defined template, we ensure that the augmented descriptions provided by LLMs have low noise and high relevance. This alignment guarantees consistency and clarity in the dataset to use later for downstream training and evaluation tasks. Moreover, using an official format greatly diminishes the variability of generated data, making it much easier to closely match the characteristics of the

Table 2
Comparison of CVE descriptions before and after the application of LLMs.

ID	Original Description	GPT3.5-Turbo	Claude3 Haiku	Claude1.2 Instant
CVE-2019-17101	Improper Neutralization of Special Elements used in a Command ('Command Injection') vulnerability in firmware versions prior to x.xx of Netatmo Smart Indoor Camera allows an attacker to execute commands on the device. This issue affects: the Netatmo Smart Indoor Camera version and prior versions.	COMMAND INJECTION in Netatmo Smart Indoor Camera firmware versions prior to x.xx allows an attacker to execute commands on the device. This vulnerability can be exploited by an attacker to run arbitrary commands on the affected device, potentially leading to unauthorized access, data theft, or further compromise of the device. The attack vector for this vulnerability involves sending specially crafted commands to the device.	Improper Neutralization of Special Elements used in a Command ('Command Injection') vulnerability in firmware versions prior to x.xx of Netatmo Smart Indoor Camera causes remote code execution on the device when an attacker sends specially crafted input.	Here is a formatted CVE description based on the provided information: [DESCRIPTION] Improper Neutralization of Special Elements used in a Command ('Command Injection') in the web interface of Netatmo Smart Indoor Camera versions before x.xx allows a remote attacker to execute arbitrary commands on the affected device. An attacker could exploit this vulnerability by submitting crafted HTTP requests containing operating system commands to the web interface. Successful exploitation could allow the attacker to manipulate and potentially compromise the device, resulting in loss of confidentiality, integrity, and availability. This vulnerability affects Netatmo Smart Indoor Camera before version x.xx.

original dataset while preserving the accuracy and utility of the refined descriptions.

The original descriptions have the longest sentences with a maximum of 408 words, whereas the descriptions generated by GPT3.5-Turbo have the shortest with 242 words. In contrast, there is a notable variation in the minimum sentence length across the four text types. The original text and C3Haiku have the longest sentences with 15 and 30 words, respectively, whereas C1Instant contains sentences with merely 3 words and GPT3.5-Turbo results in a minimum length of 11 words. The overall trend indicates that the original descriptions consist of longer and potentially more complex sentences, in comparison to the remaining three text types. Additionally, C3Haiku utilizes comparatively extended sentences, while C1Instant and GPT3.5-Turbo typically consist of less lengthy and possibly more concise sentences. Table 2 reports sample vulnerability descriptions, before and after using the LLMs.

We also performed an extensive data-cleaning process to ensure the quality and reliability of the data. The primary objective revolved around incorporating the most recent vulnerabilities documented by NVD between the years 2016 and 2024. This dataset strictly conforms to the vulnerability assessment standards delineated in versions 3.0 and 3.1. By establishing a robust groundwork, this dataset facilitates precise vulnerability evaluations and exploration of vulnerability affects various contexts. The dataset employed is refined and equitable, effectively mirroring the probability distributions of metrics and labels while accurately representing the fundamental elements of the original dataset.

4.2. Preprocessing pipeline and quality assessment

To reduce bias and noise in vulnerability descriptions, we designed a multi-stage preprocessing pipeline primarily aimed at eliminating any potential sources of label leakage. In this stage, all indicators, codes, or phrases that could directly or indirectly reveal the final classification labels were identified and removed. In addition, the texts were refined to ensure structural consistency, completeness, and the absence of duplicate or contradictory content, resulting in standardized and homogeneous inputs. For quality assessment of the GPT-3.5 Turbo-generated outputs, we adopted a systematic approach in which multiple random samples of the data were manually reviewed by two independent evaluators and compared against authoritative reference descriptions to ensure accuracy, clarity, and neutrality.

4.3. Quality of results

The experimented BERT models were fine-tuned on the complete samples of the original descriptions (137,546 data samples), with regard to the lexical content and tokenization purposes. Table 3 summarizes the results obtained.

The results show that BERT has good performance on all datasets, resulting in noticeably higher accuracy values across different vulnerability metrics. Moreover, BERT's results on augmented datasets are comparable to the original dataset. This suggests that data refinement and the generation of vulnerability descriptions based on the official CVE template can have a positive impact on the classification performance of the model. For generated datasets, GPT3.5-Turbo and C3 Haiku achieve the highest accuracy values for 4 out of 8 vulnerability metrics (AV, AC, PR, and C for GPT3.5-Turbo and UI, S, I, and A for C3 Haiku). While GPT3.5-Turbo has, on average, higher precision and F1-score values, the C3 Haiku dataset results in higher values for recall. By contrast, the C1 Instant dataset yields lower performance on the BERT model, compared to the LLM-based datasets. In comparison to BERT, DistilBERT provides varied results on different datasets, while being generally outperformed by BERT across datasets, vulnerability metrics, and evaluation metrics. This model achieves the highest accuracy, recall, F1-score, and Cohen Kappa values for the majority of vulnerability metrics when used on GPT3.5-Turbo datasets.

BERTsmall generally outperforms DistilBERT across all datasets and metrics. Despite the marginal improvement achieved by this model in some vulnerability metric, such as in I and A on the original dataset, the results provided by BERTsmall are comparatively lower than those provided by BERT. However, the model is able to predict different vulnerability metrics accurately when it is applied to LLM-generated data. It can be argued that BERTsmall might be unable to capture the complexities of the task, compared to the BERT, while being more accurate than DistilBERT. Naturally, BERTsmall has a significantly lower number of parameters than BERT, which results in relatively lower accuracy scores. Even so, this model outperforms the other models, such as TinyBERT and DistilBERT, over different datasets and metrics. Furthermore, BERTsmall has a relatively similar performance, compared to MediumBERT, while being smaller with regard to the number of model parameters.

In contrast, TinyBERT shows higher accuracy on the GPT3.5-Turbo dataset, compared to the original descriptions. In this respect, C1 Instant and C3 Haiku yield lower results in different classification metrics, on various vulnerability metrics. This suggests that these LLMs may not be effective in accurately predicting the values when used on TinyBERT. Likewise, MediumBERT produced the most accurate results on the GPT3.5-Turbo dataset, compared to the original descriptions, C1 Instant, and C3 Haiku. C3 Haiku outperformed the GPT3.5-Turbo dataset on the vulnerability metric A. MediumBERT, has higher accuracy compared to BERT, TinyBERT, and DistilBERT, and marginally lower compared to BERTsmall on the original dataset. However, the model on the generated descriptions has mixed results, providing higher accurate prediction on several metrics, and lower on others. Notably, the performance of MediumBERT is constantly higher than DistilBERT across different metrics and LLM-based generated datasets.

Table 3
Comparison of fine-tuned BERT models on original and generated datasets. A, R, P, F1, and C mean accuracy, recall, precision, F1-score and Cohen's Kappa, respectively. Best values in bold.

	Original Data					GPT3.5-Turbo					C3 Haiku					C1 Instant				
	BERT																			
	A	R	P	F1	C	A	R	P	F1	C	A	R	P	F1	C	A	R	P	F1	C
AV	0.904	0.654	0.793	0.698	0.738	**0.954**	0.849	0.934	**0.887**	**0.880**	0.950	**0.895**	0.865	0.845	0.868	0.935	0.832	0.868	0.844	0.837
AC	0.943	0.722	0.864	0.773	**0.548**	**0.968**	0.833	**0.920**	**0.870**	0.740	0.966	**0.926**	0.868	0.826	0.737	0.967	0.827	0.918	0.866	0.733
PR	0.814	0.684	0.721	0.701	0.616	**0.899**	0.825	**0.875**	**0.847**	**0.797**	0.883	**0.860**	0.833	0.811	0.758	0.858	0.708	0.829	0.745	0.700
UI	0.901	0.894	0.893	0.893	0.787	0.957	0.951	**0.955**	0.953	0.906	**0.958**	**0.952**	0.954	**0.955**	**0.908**	0.940	0.934	0.934	0.934	0.868
S	0.963	0.904	0.946	0.923	0.847	0.976	0.954	0.956	0.955	0.910	**0.980**	**0.969**	**0.962**	**0.956**	**0.925**	0.968	0.930	0.955	0.942	0.884
C	0.84	0.788	0.837	0.809	0.711	**0.902**	0.872	**0.902**	**0.886**	**0.825**	0.890	**0.896**	0.876	0.860	0.805	0.859	0.811	0.865	0.834	0.747
I	0.87	0.848	0.875	0.859	0.787	0.903	0.895	0.902	0.898	0.842	**0.918**	**0.919**	**0.911**	**0.904**	**0.865**	0.890	0.885	0.883	0.884	0.820
A	0.902	0.621	0.714	0.632	0.798	0.928	0.737	**0.871**	0.779	0.852	**0.932**	**0.854**	0.845	**0.836**	**0.862**	0.922	0.732	0.855	0.770	0.843
	DistilBERT																			
	A	R	P	F1	C	A	R	P	F1	C	A	R	P	F1	C	A	R	P	F1	C
AV	0.879	0.423	**0.936**	0.427	0.689	**0.908**	**0.731**	0.782	**0.752**	**0.774**	0.896	0.660	0.812	0.716	0.709	0.897	0.669	0.811	0.717	0.737
AC	0.916	0.500	**0.958**	0.478	0.000	**0.945**	**0.652**	0.900	**0.712**	0.431	0.940	0.609	0.927	0.661	0.334	0.932	0.616	0.871	0.666	0.343
PR	0.761	0.508	0.811	0.492	0.468	0.824	0.693	0.740	0.713	0.629	**0.841**	0.732	0.791	**0.757**	**0.668**	0.799	0.650	**0.767**	0.689	0.584
UI	0.651	0.502	0.825	0.399	0.006	**0.909**	**0.892**	**0.908**	**0.899**	**0.797**	0.900	0.882	0.898	0.889	0.778	0.892	0.887	0.884	0.885	0.771
S	**0.841**	0.500	**0.920**	**0.457**	0.000	0.823	0.500	0.911	0.451	0.000	0.838	0.500	0.919	0.456	0.000	0.827	0.500	0.913	0.453	0.000
C	0.578	0.333	**0.859**	0.244	0.000	**0.851**	**0.807**	0.850	**0.826**	**0.732**	0.845	0.805	0.837	0.819	0.724	0.832	0.769	0.841	0.798	0.692
I	0.504	0.333	0.835	0.223	0.000	0.864	0.847	0.872	0.858	0.777	**0.878**	**0.860**	**0.887**	**0.872**	**0.802**	0.870	0.847	0.877	0.859	0.784
A	0.589	0.333	0.863	0.247	0.000	**0.896**	0.619	**0.931**	0.626	**0.790**	0.875	0.616	0.915	0.625	0.749	0.878	**0.620**	0.919	**0.639**	0.755
	BERTsmall																			
	A	R	P	F1	C	A	R	P	F1	C	A	R	P	F1	C	A	R	P	F1	C
AV	0.902	0.688	0.768	0.716	0.739	**0.949**	0.814	**0.926**	0.858	**0.870**	0.941	**0.875**	0.897	**0.882**	0.855	0.936	0.824	0.848	0.834	0.840
AC	0.942	0.745	0.863	0.790	0.581	**0.966**	**0.790**	**0.933**	**0.845**	**0.690**	0.962	0.772	0.930	0.830	0.661	0.954	0.745	0.871	0.792	0.586
PR	0.815	0.716	0.737	0.724	0.616	0.868	0.759	0.846	0.795	0.720	**0.880**	**0.791**	**0.857**	**0.817**	**0.754**	0.848	0.756	0.810	0.780	0.685
UI	0.921	0.914	0.912	0.913	0.826	0.935	0.926	0.930	0.928	0.856	**0.939**	**0.927**	**0.936**	**0.931**	**0.863**	0.925	0.917	0.919	0.918	0.836
S	0.955	0.908	0.923	0.915	0.830	**0.972**	**0.944**	0.957	**0.950**	**0.901**	0.969	0.922	**0.959**	0.939	0.879	0.961	0.927	0.939	0.933	0.866
C	0.841	0.812	0.818	0.815	0.718	**0.891**	0.847	**0.895**	**0.868**	**0.799**	0.885	**0.854**	0.882	0.867	0.794	0.841	0.795	0.850	0.818	0.717
I	0.878	0.865	0.882	0.873	0.797	**0.895**	0.880	**0.901**	**0.889**	**0.827**	0.892	**0.883**	0.895	0.888	0.824	0.882	0.877	0.881	0.879	0.806
A	0.915	0.750	0.768	0.758	0.833	**0.928**	0.715	0.869	0.755	0.853	**0.928**	**0.790**	**0.910**	**0.833**	**0.856**	0.911	0.740	0.844	0.776	0.817
	TinyBERT																			
	A	R	P	F1	C	A	R	P	F1	C	A	R	P	F1	C	A	R	P	F1	C
AV	0.878	0.454	0.800	0.465	0.679	0.935	**0.850**	0.847	0.841	**0.842**	**0.940**	0.819	**0.883**	**0.848**	**0.842**	0.925	0.726	0.811	0.727	0.807
AC	0.947	0.706	0.846	0.755	0.512	**0.966**	0.821	0.937	**0.868**	0.737	0.962	**0.839**	0.885	0.860	0.720	0.962	0.766	0.895	0.816	0.633
PR	0.783	0.616	0.679	0.636	0.560	**0.881**	**0.784**	**0.859**	**0.814**	**0.748**	0.862	0.780	0.831	0.803	0.716	0.857	0.766	0.830	0.793	0.705
UI	0.887	0.863	0.889	0.873	0.747	**0.940**	0.932	**0.934**	0.933	0.866	**0.940**	**0.936**	**0.934**	**0.935**	**0.869**	0.929	0.920	0.924	0.922	0.844
S	0.957	0.900	0.945	0.920	0.841	**0.979**	**0.948**	**0.973**	**0.960**	**0.919**	0.969	0.923	0.969	0.944	0.888	0.969	0.915	0.953	0.933	0.866
C	0.809	0.757	0.799	0.775	0.656	**0.900**	**0.868**	**0.899**	**0.882**	**0.820**	0.894	0.858	0.892	0.874	0.806	0.863	0.821	0.869	0.842	0.756
I	0.865	0.851	0.864	0.857	0.782	**0.906**	**0.892**	**0.910**	**0.900**	**0.846**	0.893	0.880	0.904	0.891	0.825	0.885	0.871	0.888	0.879	0.808
A	0.907	0.635	0.739	0.647	0.808	0.908	0.701	0.799	0.731	0.812	**0.916**	**0.712**	**0.908**	**0.758**	**0.832**	0.908	0.674	0.816	0.705	0.814
	BERT Medium																			
	A	R	P	F1	C	A	R	P	F1	C	A	R	P	F1	C	A	R	P	F1	C
AV	0.907	0.637	**0.904**	0.710	0.758	**0.958**	0.857	0.877	0.866	**0.898**	0.947	**0.901**	0.894	**0.896**	0.871	0.939	0.777	0.774	0.773	0.846
AC	0.951	0.696	0.929	0.762	0.529	**0.973**	**0.837**	**0.958**	**0.887**	**0.774**	0.969	0.807	0.944	0.861	0.723	0.969	0.817	0.947	0.869	0.739
PR	0.834	0.695	0.754	0.720	0.646	**0.873**	0.784	0.799	0.789	0.732	0.870	**0.821**	**0.825**	**0.823**	**0.734**	0.855	0.760	0.809	0.781	0.699
UI	0.906	0.889	0.905	0.896	0.793	0.941	0.935	0.937	0.936	0.873	**0.952**	**0.940**	**0.953**	**0.946**	0.892	0.929	0.914	0.930	0.921	0.842
S	0.963	0.915	0.951	0.932	0.863	**0.974**	**0.948**	**0.969**	**0.958**	**0.917**	**0.974**	0.944	0.968	0.955	0.911	0.960	0.922	0.949	0.935	0.869
C	0.834	0.803	0.812	0.806	0.707	**0.889**	**0.857**	**0.893**	**0.873**	**0.805**	0.884	**0.857**	0.869	0.863	0.793	0.864	0.829	0.861	0.844	0.756
I	0.852	0.835	0.836	0.836	0.754	**0.896**	**0.892**	0.887	0.889	0.831	0.902	0.885	**0.907**	**0.895**	**0.839**	0.878	0.869	0.879	0.873	0.800
A	0.903	0.708	0.820	0.742	0.805	**0.926**	0.674	0.786	0.702	0.843	0.917	**0.750**	**0.942**	**0.803**	0.832	0.923	0.706	0.817	0.737	**0.845**

Overall, BERT appears to perform well across the different datasets. DistilBERT shows promising performance on specific generated datasets, but is outperformed by other models. On the other hand, BERTsmall achieves reasonable performance on both the original and generated datasets, closely following BERT. TinyBERT struggles to produce accurate predictions with C1 Instant and C3 Haiku, having higher accuracy with GPT3.5-Turbo. MediumBERT also results in mixed outcomes across datasets, and compared to different models. Importantly, C1 Instant generated descriptions appear to be the least effective, as they yield very minor improvement in the models' performance.

To evaluate errors, we first calculate the CVSS score for all CVEs in the datasets based on the predicted metrics for each vulnerability by using the CVSS V3.1 calculator's formula.[4] Then, we calculate MSE and MAE based on the calculated CVSS score and the real CVSS score. The Base Score is defined as follows:

- If Impact ≤ 0, then Base Score $= 0$.
- Otherwise, if the scope is unchanged, then Base Score $= \lceil \min(\text{Impact} + \text{Exploitability}, 10) \rceil$.
- Otherwise, Base Score $= \lceil \min(1.08 \times (\text{Impact} + \text{Exploitability}), 10) \rceil$.

where Impact is defined as:

[4] https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator

Table 4
MSE and MAE obtained. Best values in bold.

	GPT-3.5Turbo		C3 Haiku		C1 Instant	
	MSE	MAE	MSE	MAE	MSE	MAE
BERT	0.3394	**0.16**	0.4135	0.2061	0.6019	**0.3158**
DistilBERT	0.8319	0.4868	0.8604	0.5239	0.9326	0.5628
BERTsmall	**0.2996**	0.1664	0.462	0.2243	**0.652**	0.3335
TinyBERT	0.3468	0.1725	0.5903	0.276	0.7498	0.3555
MediumBERT	0.4469	0.2036	**0.365**	**0.1706**	0.7495	0.3208

- If the scope is unchanged, then Impact = 6.42× ISS;
- Otherwise, Impact = 7.52 × (ISS−0.029) − 3.25 × (ISS −0.02)[15];

with ISS $= 1 - [(1 - \text{Impact}_{\text{Conf}}) \times (1 - \text{Impact}_{\text{Integ}}) \times (1 - \text{Impact}_{\text{Avail}})]$, and Exploitability is defined as 8.22× AV × AC × PR × UI.

Table 4 reports MSE and MAE for each BERT model using each LLM. BERTsmall results in the lowest MSE, while DistilBERT has by far the highest. BERT, TinyBERT, and MediumBERT stay at a mid-range MSE of over 0.3, and MediumBERT has the highest error of over 0.4 on the GPT3.5-Turbo descriptions. MAE follows a quite similar trend, according to which, BERTsmall, TinyBERT, and BERT result in around a value of 0.16. By contrast, DistilBERT results in the highest MAE error among the models, and MediumBERT sees a marginally higher value of 0.2. Overall, BERT and BERTsmall appear to have a good balance of MSE and MAE on the data generated by GPT3.5-Turbo, while DistilBERT has the highest MSE and MAE.

Likewise, DistilBERT has the highest MAE and MSE values on the C1 Instant descriptions. Both error metrics have a huge increase compared to the GPT3.5-Turbo and C3 Haiku-generated descriptions. In particular, the results associated with BERT, BERTsmall, TinyBERT, and MediumBERT exhibit approximately double MAE values on the GPT3.5-Turbo data. DistilBERT shows almost consistent error values of between 0.8 and 0.9 across all generated datasets. Compared to C1 Instant, C3 Haiku seems to produce more informative and effective descriptions, in particular with BERT, BERTsmall, and MediumBERT. MediumBERT seems to be the most effective. BERT results in relatively similar results, with marginally higher error values. DistilBERT still yields the worst results. Notably, MediumBERT achieves slightly lower error values on the C3 Haiku dataset in comparison to GPT3.5-Turbo. TinyBERT and BERTsmall, by contrast, reach a noticeably higher MSE when C3 Haiku descriptions are used rather than GPT3.5-Turbo descriptions.

4.4. Impact of LLMs

We now "zoom in" on the prediction accuracy and precision obtained when generating descriptions using the three different LLMs.

Table 5 summarizes the results obtained with C1 Instant. Regarding accuracy, all models could reach relatively high accuracy values (over 80 % for all the vulnerability metrics). BERT and MediumBERT achieved the highest values, closely followed by TinyBERT and BERTsmall. DistilBERT, on the other hand, resulted in the lowest accuracy. While BERT outperforms other models on PR, UI, and I, MediumBERT achieves the highest accuracy on AC, AV, C, and A — TinyBERT showed marginally superior performance on S. In comparison to the accuracy values, precision scores are generally lower across all models and vulnerability metrics. For instance, while BERT achieves the highest precision for AV, UI, and S, TinyBERT outperforms the other models on PR, C, and I. Similarly, MediumBERT and DistilBERT show superior results on AC and A, respectively. Notably, BERTsmall has mid-range precision scores on all the metrics, outperforming TinyBERT in AV, and A. Consequently, BERT and MediumBERT seem to be more balanced and accurate than the others. On the other hand, the findings on C1 Instant indicate that BERT shows very good performance across vulnerability metrics, specifically recall and F1-score. BERT is closely followed by TinyBERT, MediumBERT, and BERTsmall, while DistilBERT has the worst performance,

Table 5
Classification metrics obtained with C1 Instant. Best values in bold.

C1 Instant (Accuracy)								
	AV	AC	PR	UI	S	C	I	A
BERT	0.935	0.967	**0.858**	**0.940**	0.968	0.859	**0.890**	0.922
DistilBERT	0.897	0.932	0.799	0.892	0.827	0.832	0.870	0.878
BERTsmall	0.936	0.954	0.848	0.925	0.961	0.841	0.882	0.911
TinyBERT	0.925	0.962	0.857	0.929	**0.969**	0.863	0.885	0.908
MediumBERT	**0.939**	**0.969**	0.855	0.929	0.960	**0.864**	0.878	**0.923**
C1 Instant (Precision)								
	AV	AC	PR	UI	S	C	I	A
BERT	**0.868**	0.918	0.829	**0.934**	**0.955**	**0.865**	0.883	0.855
DistilBERT	0.811	0.871	0.767	0.884	0.913	0.841	0.877	**0.919**
BERTsmall	0.848	0.871	0.810	0.919	0.939	0.850	0.881	0.844
TinyBERT	0.811	0.895	**0.830**	0.924	0.953	0.869	**0.888**	0.816
MediumBERT	0.774	**0.947**	0.809	0.930	0.949	0.861	0.879	0.817
C1 Instant (Recall)								
	AV	AC	PR	UI	S	C	I	A
BERT	**0.832**	**0.827**	0.708	**0.934**	**0.930**	0.811	**0.885**	0.732
DistilBERT	0.669	0.616	0.650	0.887	0.500	0.769	0.847	0.620
BERTsmall	0.824	0.745	0.756	0.917	0.927	0.795	0.877	**0.740**
TinyBERT	0.726	0.766	**0.766**	0.920	0.915	0.821	0.871	0.674
MediumBERT	0.777	0.817	0.760	0.914	0.922	**0.829**	0.869	0.706
C1 Instant (F1-score)								
	AV	AC	PR	UI	S	C	I	A
BERT	**0.844**	0.866	0.745	**0.934**	**0.942**	0.834	**0.884**	**0.770**
DistilBERT	0.717	0.666	0.689	0.885	0.453	0.798	0.859	0.639
BERTsmall	0.834	0.792	0.780	0.918	0.933	0.818	0.879	0.776
TinyBERT	0.727	0.816	**0.793**	0.922	0.933	0.842	0.879	0.705
MediumBERT	0.773	**0.869**	0.781	0.921	0.935	**0.844**	0.873	0.737
C1 Instant (Cohen's Kappa Score)								
	AV	AC	PR	UI	S	C	I	A
BERT	0.837	0.733	0.700	**0.868**	**0.884**	0.747	**0.820**	0.843
DistilBERT	0.737	0.343	0.584	0.771	0.000	0.692	0.784	0.755
BERTsmall	0.840	0.586	0.685	0.836	0.866	0.717	0.806	0.817
TinyBERT	0.807	0.633	**0.705**	0.844	0.866	**0.756**	0.808	0.814
MediumBERT	**0.846**	**0.739**	0.699	0.842	0.869	**0.756**	0.800	**0.845**

especially on S and AC. This tells us that the smaller models, namely BERTsmall and TinyBERT, offer a nice balance between accuracy and model size, with reasonable results across most categories.

Table 6 summarizes the results obtained with C3Haiku. Overall, all models provide good performance. BERT shows superior performance on AV, UI, S, I, and A, in terms of accuracy. In comparison, MediumBERT, BERTsmall, and TinyBERT achieve the highest accuracy on AC, PR, and C, respectively — the results appear fairly similar to those obtained with C1 Instant. Whereas MediumBERT provides the highest precision values for AC and A, BERT shows superior performance on UI, and I. BERTsmall and TinyBERT result in the highest precision for (AV, PR) and (S, C), respectively. C3 Haiku shows a more positive impact on the performance of the models, increasing the accuracy and precision of the majority of the metrics. Moreover, similar to the case of C1 Instant, BERT and MediumBERT show promising results on vulnerability prediction. Similarly to the results on C1 Instant, BERT shows a superior performance compared to the other models, in terms of Recall, F1-score, and Cohen's Kappa. MediumBERT and TinyBERT showed moderate performance, while DistilBERT and BERTsmall seem to struggle in recall, F1-score, and Cohen's Kappa.

Finally, Table 7 summarizes the results obtained with GPT3.5-Turbo. GPT3.5-Turbo seems to provide higher accuracy scores with respect to the other LLMs while resulting in slightly lower precision in several vulnerability metrics. Similar to the Claude-based datasets, precision is generally lower than accuracy for all the models. Different models still maintain good accuracy overall. In particular, BERT and

Table 6
Classification metrics obtained with C3 Haiku. Best values in bold.

C3 Haiku (Accuracy)								
	AV	AC	PR	UI	S	C	I	A
BERT	**0.950**	0.966	**0.883**	**0.958**	**0.980**	0.890	**0.918**	**0.932**
DistilBERT	0.896	0.940	0.841	0.900	0.838	0.845	0.878	0.875
BERTsmall	0.941	0.962	0.880	0.939	0.969	0.885	0.892	0.928
TinyBERT	0.940	0.962	0.862	0.940	0.969	**0.894**	0.893	0.916
MediumBERT	0.947	**0.969**	0.870	0.952	0.974	0.884	0.902	0.917
C3 Haiku (Precision)								
	AV	AC	PR	UI	S	C	I	A
BERT	0.865	0.868	0.833	**0.954**	0.962	0.876	**0.911**	0.845
DistilBERT	0.812	0.927	0.791	0.898	0.919	0.837	0.887	0.915
BERTsmall	**0.897**	0.930	**0.857**	0.936	0.959	0.882	0.895	0.910
TinyBERT	0.883	0.885	0.831	0.934	**0.969**	**0.892**	0.904	0.908
MediumBERT	0.894	**0.944**	0.825	0.953	0.968	0.869	0.907	**0.942**
C3 Haiku (Recall)								
	AV	AC	PR	UI	S	C	I	A
BERT	0.895	**0.926**	**0.860**	**0.952**	**0.969**	**0.896**	**0.919**	**0.854**
DistilBERT	0.660	0.609	0.732	0.882	0.500	0.805	0.860	0.616
BERTsmall	0.875	0.772	0.791	0.927	0.922	0.854	0.883	0.790
TinyBERT	0.819	0.839	0.780	0.936	0.923	0.858	0.880	0.712
MediumBERT	**0.901**	0.807	0.821	0.940	0.944	0.857	0.885	0.750
C3 Haiku (F1-score)								
	AV	AC	PR	UI	S	C	I	A
BERT	0.845	0.826	0.811	**0.955**	**0.956**	0.860	**0.904**	**0.836**
DistilBERT	0.716	0.661	0.757	0.889	0.456	0.819	0.872	0.625
BERTsmall	0.882	0.830	0.817	0.931	0.939	0.867	0.888	0.833
TinyBERT	0.848	0.860	0.803	0.935	0.944	**0.874**	0.891	0.758
MediumBERT	**0.896**	**0.861**	**0.823**	0.946	0.955	0.863	0.895	0.803
C3 Haiku (Cohen's Kappa score)								
	AV	AC	PR	UI	S	C	I	A
BERT	0.868	**0.737**	**0.758**	**0.908**	**0.925**	0.805	**0.865**	**0.862**
DistilBERT	0.709	0.334	0.668	0.778	0.000	0.724	0.802	0.749
BERTsmall	0.855	0.661	0.754	0.863	0.879	0.794	0.824	0.856
TinyBERT	0.842	0.720	0.716	0.869	0.888	**0.806**	0.825	0.832
MediumBERT	**0.871**	0.723	0.734	0.892	0.911	0.793	0.839	0.832

Table 7
Classification metrics obtained with GPT3.5-Turbo. Best values in bold.

GPT3.5-Turbo (Accuracy)								
	AV	AC	PR	UI	S	C	I	A
BERT	0.954	0.968	**0.899**	**0.957**	0.976	**0.902**	0.903	**0.928**
DistilBERT	0.908	0.945	0.824	0.909	0.823	0.851	0.864	0.896
BERTsmall	0.949	0.966	0.868	0.935	0.972	0.891	0.895	**0.928**
TinyBERT	0.935	0.966	0.881	0.940	**0.979**	0.900	**0.906**	0.908
MediumBERT	**0.958**	**0.973**	0.873	0.941	0.974	0.889	0.896	0.926
GPT3.5-Turbo (Precision)								
	AV	AC	PR	UI	S	C	I	A
BERT	**0.934**	0.920	**0.875**	**0.955**	0.956	**0.902**	0.902	0.871
DistilBERT	0.782	0.900	0.740	0.908	0.911	0.850	0.872	**0.931**
BERTsmall	0.926	0.933	0.846	0.930	0.957	0.895	0.901	0.869
TinyBERT	0.847	0.937	0.859	0.934	**0.973**	0.899	**0.910**	0.799
MediumBERT	0.877	**0.958**	0.799	0.937	0.969	0.893	0.887	0.786
GPT3.5-Turbo (Recall)								
	AV	AC	PR	UI	S	C	I	A
BERT	0.849	0.833	**0.825**	**0.951**	**0.954**	**0.872**	**0.895**	**0.737**
DistilBERT	0.731	0.652	0.693	0.892	0.500	0.807	0.847	0.619
BERTsmall	0.814	0.790	0.759	0.926	0.944	0.847	0.880	0.715
TinyBERT	0.850	0.821	0.784	0.932	0.948	0.868	0.892	0.701
MediumBERT	**0.857**	**0.837**	0.784	0.935	0.948	0.857	0.892	0.674
GPT3.5-Turbo (F1-score)								
	AV	AC	PR	UI	S	C	I	A
BERT	**0.887**	0.870	**0.847**	**0.953**	0.955	**0.886**	0.898	**0.779**
DistilBERT	0.752	0.712	0.713	0.899	0.451	0.826	0.858	0.626
BERTsmall	0.858	0.845	0.795	0.928	0.950	0.868	0.889	0.755
TinyBERT	0.841	0.868	0.814	0.933	**0.960**	0.882	**0.900**	0.731
MediumBERT	0.866	**0.887**	0.789	0.936	0.958	0.873	0.889	0.702
GPT3.5-Turbo (Cohen's Kappa score)								
	AV	AC	PR	UI	S	C	I	A
BERT	0.880	0.740	**0.797**	**0.906**	0.910	**0.825**	0.842	0.852
DistilBERT	0.774	0.431	0.629	0.797	0.000	0.732	0.777	0.790
BERTsmall	0.870	0.690	0.720	0.856	0.901	0.799	0.827	**0.853**
TinyBERT	0.842	0.737	0.748	0.866	**0.919**	0.820	**0.846**	0.812
MediumBERT	**0.898**	**0.774**	0.732	0.873	0.917	0.805	0.831	0.843

MediumBERT showed similar superior performance on (PR, UI, C, A) and (AV, AC), while BERTsmall and TinyBERT outperformed the other models on A and (S, I), respectively. The precision scores support the above claims regarding the BERT model, as it shows superior performance on the majority of the vulnerability metrics, including AV, PR, UI, and C. MediumBERT had the highest precision for AC, and DistilBERT and TinyBERT outperformed the others on A and (S, I), respectively. In comparison, BERTsmall maintains a relatively high precision score over different metrics. BERT and MediumBERT outperformed the smaller models across different vulnerability metrics. Looking at recall and Cohen's Kappa, the smaller models showed relatively worse performance, and in particular, DistilBERT struggled in both classification metrics.

In summary, the descriptions generated using GPT3.5-Turbo resulted in superior performance — C3 Haiku is the second best. BERTsmall and MediumBERT exhibit high scores across different metrics when using C3 Haiku.

4.5. Computational costs

Table 8 reports computation times for each BERT models and each LLM. Specifically, we report the average training time required for a single epoch (in minutes). The computation time appears to be influenced by both the model size and the complexity of the dataset. Smaller models, such as TinyBERT, BERTsmall, and MediumBERT, have generally lower computation times than larger ones. For instance, TinyBERT takes 0.26 min to process the GPT3.5-Turbo dataset, while BERT takes around 1 min. Likewise, it can be seen that BERTsmall has very similar computation times as TinyBERT. GPT3.5-Turbo exhibits the lower times with respect to all the models. BERT and BERTsmall have similar performance. The results suggest that in those cases where larger models, such as BERT, cannot be used, and there is a requirement for more lightweight, yet accurate models, *the combination of the GPT3.5-Turbo LLM and BERTsmall can efficiently produce precise predictions*.

Table 8
Computation times (minutes per epoch). Best values in bold.

	GPT3.5-Turbo	**C1 Instant**	**C3 Haiku**
TinyBERT	**0.26**	0.4	0.45
BERTsmall	**0.28**	0.42	0.47
MediumBERT	**0.35**	0.5	0.55
DistilBERT	**0.8**	1.0	1.0
BERT	**1.0**	1.2	1.25

4.6. Model efficiency analysis

To evaluate the computational efficiency of the proposed architecture, we have measured each BERT model across the original NVD, GPT3.5-Turbo, C3 Haiku, and C1 Instant datasets as shown in Table 9, reporting their FLOPs, number of parameters, and average inference time. As the findings show, TinyBERT showed the most efficiency, having the least FLOPs (0.044-0.082 GFLOPs), parameters (0.413 M),

Table 9
Comparison of different BERT models regarding their FLOPs, parameters, and average inference time, measured in GFLOPS (G), Million (M), and Seconds (S), respectively.

	Original			GPT3.5-Turbo			C3 Haiku			C1 Instant		
	FLOPs (G)	Params (M)	Time (S)	FLOPs (G)	Params (M)	Time (S)	FLOPs (G)	Params (M)	Time (S)	FLOPs (G)	Params (M)	Time (S)
BERT	14.512	85.647	8.853	9.369	85.647	8.711	14.521	85.647	8.883	17.693	85.647	8.712
DistilBERT	14.512	85.647	8.868	9.369	85.647	8.79	14.521	85.647	8.847	17.693	85.647	8.635
MediumBERT	4.302	25.483	6.206	2.777	25.483	6.067	4.305	25.483	6.207	5.245	25.483	6.2
BERTsmall	2.151	12.873	3.565	1.389	12.873	3.497	2.153	12.873	3.571	2.623	12.873	3.582
TinyBERT	0.068	0.413	2.308	0.044	0.413	2.185	0.068	0.413	2.297	0.082	0.413	2.312

Table 10
Comparison with [12]. Best values in bold.

	[12]					Proposed Architecture				
	A	R	P	F1	C	A	R	P	F1	C
AV	0.91	**0.82**	0.82	0.82	0.81	**0.949**	0.814	**0.926**	**0.858**	**0.870**
AC	0.96	**0.80**	0.84	0.83	0.61	**0.966**	0.790	**0.933**	**0.845**	**0.690**
PR	0.86	**0.80**	0.81	**0.80**	**0.73**	**0.868**	0.759	**0.846**	0.795	0.720
UI	0.93	**0.94**	**0.94**	**0.94**	**0.88**	**0.935**	0.926	0.930	0.928	0.856
S	0.95	0.91	0.93	0.92	0.85	**0.972**	**0.944**	**0.957**	**0.950**	**0.901**
C	0.89	0.85	0.86	**0.87**	**0.80**	**0.891**	0.847	**0.895**	0.868	0.799
I	0.88	0.87	0.89	0.87	0.81	**0.895**	**0.880**	**0.901**	**0.889**	**0.827**
A	0.90	**0.72**	0.80	0.75	0.80	**0.928**	0.715	**0.869**	**0.755**	**0.853**
MSE			1.52					**0.299**		
MAE			0.624					**0.166**		

and inference times (2.185-2.3125 S) of models, making it ideal for environments under resource constraints. Although, it did not come as a fully adopted model because it traded off accuracy with that of larger models. Closely following, BERTsmall provides a balanced trade-off with FLOPs of 1.389-2.623 GFLOPs, parameters of 12.873 M, and inference times of 3.4975-3.5825 S, all the while maintaining high accuracy across a range of metrics, on GPT3.5-Turbo. It does this while having noticeably reduced computation overhead compared to BERT and DistilBERT. MediumBERT, on the other hand, is fairly moderate at 25.483 million parameters and has FLOPs that range from 2.777 to 5.245 GFLOPs. Meanwhile, BERT and DistilBERT produce quite lengthy inferences (8.635-8.88375 S) with large comparable parameters of 85.647 million but higher FLOPs (9.369-17.693 GFLOPs). The Low FLOPs and inference time conditions of the GPT3.5-Turbo dataset is apparent across all models, therefore making it the most efficient when used to generate short and template-conforming descriptions having very low computational requirements with high predictive performance, especially when partnered with BERTsmall.

4.7. Comparison with the state of the art

We compared the performance of our proposed architecture with the current state-of-the-art [12], whose authors integrate data from 66 resources (including the NVD dataset) to improve the performance of automated vulnerability scoring. We only used the NVD dataset and GPT3.5-Turbo to augment it. The results obtained using BERTsmall are reported in Table 10. Our proposed architecture significantly improves performance across most metrics. The significant reduction in MSE and MAE indicates a closer fit to actual vulnerability scores, suggesting better accuracy and reliability in predictions. Our method generally achieves comparable or better recall and precision, indicating a better balance — to confirm this, the F1-score consistently shows higher values. Cohen's Kappa scores are also generally better, reflecting a stronger agreement between predicted and actual vulnerability scores. A significant reduction in both MSE and MAE compared indicates fewer large errors in predictions and smaller average error magnitudes. The results highlight the importance of data quality and superior performance of our proposed architecture compared with state-of-the-art methods.

4.8. Robustness test

To demonstrate the robustness of the proposed architecture, experimental tests were carried out to evaluate model performance under noisy input conditions. Such noise includes two types of distortions, namely 1) character-level distortions (random swaps or replacements of characters within words through different character letters) and 2) semantic-level distortions (randomly substituting words with their synonyms according to WordNet). Each noise type contains an additional four different noise levels: 10 %, 20 %, 30 %, and 40 %. Such percentage refers to the number of words modified in each sample. This fine-tuned model was tested on each noisy dataset, the results of which are summarized in Table 11, where character and semantic noise are denoted by *Char* and *Sem*, respectively, and demonstrating the robustness of the model to various resistances and kinds of noise.

The results in Table 11 show that the model GPT-3.5 Turbo + BERTsmall maintained its effectiveness under both character and semantic noise, although we can observe more accuracy degradation under increasing degrees of noise. For example, at 10 to 40 % noise increase, AV, PR, C, I, and A metrics decline noticeably in character-level noise, decreasing from 0.949 to 0.851 for AV, or from 0.868 to 0.748 for PR at 40 % noise. In contrast, AC and S remain relatively stable, with AC dropping slightly from 0.966 to 0.94 and S from 0.972 to 0.946, hinting those metrics are less subject to character perturbations. Semantic-level noise (synonym substitution) do not generally cause major degradation, as PR even slightly improves at 10 % noise (0.878 vs. 0.868) and all other metrics remain above 0.8 most of the time for 40 % noise (e.g., AV at 0.907, S at 0.956). This is likely because the model was fine-tuned on semantically rich, template-adhering descriptions generated by GPT-3.5 Turbo so that the model would generalize over similar terms-the kind of effect shown in robust studies about NLP models. Notably, the model's robustness differs across different metrics. Metrics like AC and S which rely on broader contextually understanding synonyms tend to have very little drop in performance metrics, while PR and C which rely heavily on words are more affected by the noise. A major drop in accuracy for AV, PR, C, I, and A at character-level noise of 40 % indicates that beyond that, the architecture may have limits in handling extreme distortions, especially when key tokens have been substantially altered. All in all, these results confirm the architecture's functioning under more realistic noise levels (10-20 %) in vulnerability descriptions, thus reinforcing its practical applicability in the cybersecurity field with further directions for improvement under extreme perturbations.

5. Discussion and limitations

The results obtained show that, while all BERT variations reach very good accuracy across various vulnerability metrics, the issue of balancing accuracy and computation load should be taken into account. Moreover, each of the models outperformed the others on certain vulnerability and accuracy metrics. Despite achieving higher precision and recall, larger models like BERT and MediumBERT fell slightly short in accuracy and precision compared to smaller ones like TinyBERT and BERTsmall

Table 11
Robustness analysis of the proposed GPT3.5-Turbo + BertSmall architecture on different noisy inputs, in terms of accuacy.

Noise Type	No Noise	Char (10 %)	Sem (10 %)	Char (20 %)	Sem (20 %)	Char (30 %)	Sem (30 %)	Char (40 %)	Sem (40 %)
AV	**0.949**	0.94	0.947	0.913	0.94	0.891	0.918	0.851	0.907
AC	**0.966**	0.953	0.955	0.949	0.952	0.944	0.946	0.94	0.945
PR	0.868	0.861	**0.878**	0.821	0.853	0.784	0.83	0.748	0.807
UI	0.935	0.932	**0.943**	0.906	0.925	0.883	0.892	0.854	0.892
S	**0.972**	0.965	0.971	0.96	0.967	0.951	0.958	0.946	0.956
C	**0.891**	0.881	0.865	0.847	0.865	0.818	0.846	0.794	0.825
I	**0.895**	0.88	0.898	0.843	0.872	0.812	0.847	0.786	0.847
A	**0.928**	0.913	0.928	0.889	0.909	0.891	0.891	0.817	0.872

for specific components (AC and S). This highlights the need to consider different models for different metrics.

The findings also support the effectiveness of template-based descriptions in enhancing the performance of the predictive models, by providing high-quality input data that follow a unified style and language. This has been shown to improve predictions under all models, resulting in GPT3.5-Turbo (as description refiner) combined with BERTsmall (as the predictive model) to guarantee the best accuracy while also demanding much less computational time compared to the other BERT-based models.

Vulnerability assessment architectures may face some difficulties and limitations. Likewise, BERT models can be prone to some threats that may compromise their validity and reliability. The proposed architecture is no exception. In this respect, the quality of the generated data could influence the architecture's performance. If the original descriptions contain noise and biased language, the model's accuracy and understanding of the language can be affected, leading to lower-quality predictions. To mitigate this issue, the proposed architecture uses LLMs, as powerful language models, to reduce bias in data and provide higher-quality, domain-specific, and CVE-adhering descriptions. Noteworthy, despite the effectiveness of the defined template used in this study in officially formatting the descriptions, given such a standardized format may not be flexible enough to consider some of the finer, contextual details articulated in individual descriptions. This is especially for new or complex features which do not adhere strictly to the standard patterns. This could put some critical points at risk of information loss because the information would not be captured within the bound limitations of the template. Likewise, the fixed model may fail to cover a broad range of types of vulnerabilities, such as those needing deeper environmental or temporal context. In this respect, future work could analyze the possibility of adaptive or customized templates that may change dynamically concerning certain vulnerability characteristics, or those which will accommodate an added field for particular cases. This will improve generalizability and expressiveness in the forged descriptions while still retaining the features of standardization.

There are also scenarios that the proposed architecture of GPT-3.5 Turbo and BERTsmall may underperform, including in high noise, unbalanced datasets, vague descriptions, or when entirely new domain-specifying terms. Robustness test in Table 11 already shows very significant accuracies dropping from 40 % character level noise (e.g., AV becomes 0.949 to 0.851, PR from 0.868 to 0.748), though semantic noise has less impact (e.g., S from 0.972 to 0.956). An unbalanced dataset with underrepresented classes (such as rare high-severity vulnerabilities) can also bias predictions and reduce recall for minority classes. Also, ambiguous or incomplete NVD descriptions with missing key details of attack vectors, and non-standard cybersecurity terms into which the model has not been trained may lead to misclassification, especially in terms such as Confidentiality or Integrity. Future works may include adversarial training, synthetic data generation, and adding foreign knowledge bases to effect these cross domains.

Furthermore, deep learning models are inherently black-box, lacking transparency in their decision-making process. Clarity is very important in security applications, as it may help experts pick the best solutions – lack of transparency is therefore a concern that requires proper attention. To enhance the interpretability of our proposed architecture using, in particular, BERT-based models for vulnerability scoring, we can explore a number of approaches. Attention visualization will provide some insight into attention mechanisms of models like BERT across input tokens, signifying which parts of vulnerability descriptions (such as attack vectors or impact phrases) led to a prediction. In cybersecurity, this knowledge becomes important, as understanding model attention may bring insight into possible biases or patterns that have been missed in descriptions [52,53]. Explanations can also be provided by applying post-hoc explanation methods such as LIME [54] or SHAP [55], which work on the level of instances and approximate model behavior by perturbing inputs and attributing importance to features like specific keywords in CVE templates. Such explanations produce higher trustworthiness and aid debugging in vulnerability assessment settings by highlighting the terms which were influential in predicting the severity of a CVE. While such approaches could increase the transparency of our lightweight BERTsmall predictions, thereby assisting experts in validating their outputs and conforming to the processes of cybersecurity regulations, care must be taken to address vulnerabilities in these very explanations [56]. Future investigations will subject these integrations to a balancing act between efficiency and explainability.

In addition, using LLMs and generative models may raise certain ethical and privacy concerns. For privacy, we have obtained all input data from publically available data sources, which is NVD. This data source does not contain any personally identifiable information or confidential proprietary content. In addition, we have performed preprocessing privacy checks to confirm their absence in our data. Our preprocessing pipeline sought to eliminate explicit label information and contextual indications to lessen the chance of leakage to ensure generated descriptions free from potential biases in the output; those that might arise from language models replicating biases from the training data.

Overcoming the limitations discussed above is critical for real-world implementations of vulnerability assessment models, but it is not the main focus of this study. We plan to look into them in future work by, e.g., exploring more advanced data enhancement, explainable AI, and privacy-preserving techniques to develop more interpretable and secure models.

6. Conclusions and future directions

In this study, we investigated the efficacy of employing template-based vulnerability descriptions and conducted a comprehensive analysis of various LLMs for generating and refining such descriptions. We evaluated different BERT models to identify the most suitable one for categorizing vulnerability metrics. We proposed a novel methodology that employs a standardized template for vulnerability descriptions instead of the much more diverse original ones found in reported NVD data, specifically for severity assessment. Introducing a novel generative LLM-based classification model and a fine-tuned BERT tokenizer, our experiments proved that the combination of GPT3.5-Turbo and BERTsmall yields high accuracy and demands an acceptable computational load compared to other BERT-based models. These results

suggest that our proposed model, fine-tuned on initial datasets and tested on both original and template-based vulnerability descriptions, encapsulates crucial attributes for vulnerability classification, leading to significantly enhanced accuracy. The thorough examination of both original and template-based vulnerability descriptions further underscores the advantages of employing a standardized template that captures the essential information on descriptive components, without unnecessary details – the use of a well-organized template acts as a unifying factor for diverse reported data, facilitating vulnerability predictions. The utilization of a more specialized and sophisticated model tailored to the specific task at hand can substantially improve the accuracy of vulnerability classification systems.

We plan our future work in several directions:

- Assessing different template configurations and developing specific protocols for automatic vulnerability scoring.
- Incorporating a wider spectrum of vulnerability components and exploring the correlations among these components that can affect the classification process.
- Exploring more advanced data enhancement techniques, in order to better tackle data that contains noise and biased language.
- Looking into the use of explainable AI to improve the transparency of the prediction process.
- Applying privacy-preserving techniques to develop models that suitably address privacy issues.

Author Agreement Statement

We the undersigned declare that this manuscript is original, has not been published before and is not currently being considered for publica9on elsewhere. We confirm that the manuscript has been read and approved by all named authors and that there are no other persons who sa9sfied the criteria for authorship but are not listed. We further confirm that the order of authors listed in the manuscript has been approved by all of us.

We understand that the Corresponding Author is the sole contact for the Editorial process. He/she is responsible for communica9ng with the other authors about progress, submissions of revisions and final approval of proofs.

CRediT authorship contribution statement

Seyedeh Leili Mirtaheri: Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis, Data curation, Conceptualization; **Andrea Pugliese:** Writing – review & editing, Supervision, Methodology, Funding acquisition; **Valerio Pascucci:** Writing – review & editing, Validation, Methodology, Investigation.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was partially supported by project SERICS (PE00000014) under the MUR National Recovery and Resilience Plan funded by the European Union – NextGenerationEU.

References

[1] M. Corporation, CVE Assignment Information Format, 2024. Accessed: 2024-11-19, https://cve.mitre.org/cve/list_rules_and_guidance/cve_assignment_information_format.html.
[2] F. Lomio, E. Iannone, A. De Lucia, F. Palomba, V. Lenarduzzi, Just-in-time software vulnerability detection: are we there yet?, J. Syst. Softw. 188 (2022) 111283.
[3] F. Alves, A. Bettini, M.F. Pedro, A. Bessani, Processing tweets for cybersecurity threat awareness, Inf. Syst. 95 (2021) 101586.
[4] H. Chen, J. Liu, R. Liu, N. Park, V.S. Subrahmanian, VASE: a twitter-based vulnerability analysis and score engine, in: 2019 IEEE International Conference on Data Mining (ICDM) (2019) 976–981.
[5] S. Yitagesu, X. Zhang, Z. Feng, X. Li, Z. Xing, Automatic part-of-speech tagging for security vulnerability descriptions, in: 2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR), IEEE, 2021, pp. 29–40.
[6] J. Zhang, H. Bu, H. Wen, Y. Chen, L. Li, H. Zhu, When llms meet cybersecurity: a systematic literature review, arXiv:2405.03644. (2024).
[7] B. Zoph, C. Raffel, D. Schuurmans, D. Yogatama, D. Zhou, D. Metzler, E.H. Chi, J. Wei, J. Dean, L.B. Fedus, et al., Emergent abilities of large language models, TMLR (2022).
[8] S. Minaee, T. Mikolov, N. Nikzad, M. Chenaghlu, R. Socher, X. Amatriain, J. Gao, Large language models: a survey, arXiv:2402.06196. (2024).
[9] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, Y. Zhang, A survey on large language model (llm) security and privacy: the good, the bad, and the ugly, High-Confidence Comput. (2024) 100211.
[10] Y. Yigit, W.J. Buchanan, M.G. Tehrani, L. Maglaras, Review of Generative AI Methods in Cybersecurity, arXiv:2403.08701. (2024).
[11] C. Elbaz, L. Rilling, C. Morin, Fighting N-day vulnerabilities with automated CVSS vector prediction at disclosure, in: Proceedings of the 15th International Conference on Availability, Reliability and Security, 2020, pp. 1–10.
[12] P. Kühn, D.N. Relke, C. Reuter, Common vulnerability scoring system prediction based on open source intelligence information sources, Comput. Secur. 131 (2023) 103286.
[13] H. Chen, J. Liu, R. Liu, N. Park, V.S. Subrahmanian, VEST: a System for Vulnerability Exploit Scoring & Timing, in: IJCAI, 2019, pp. 6503–6505.
[14] X. Sun, Z. Ye, L. Bo, X. Wu, Y. Wei, T. Zhang, B. Li, Automatic software vulnerability assessment by extracting vulnerability elements, J. Syst. Softw. 204 (2023) 111790.
[15] Q. Zhao, C. Huang, L. Dai, VULDEFF: Vulnerability detection method based on function fingerprints and code differences, Knowl. Based Syst. 260 (2023) 110139. https://doi.org/10.1016/j.knosys.2022.110139
[16] J. Yin, M. Tang, J. Cao, H. Wang, Apply transfer learning to cybersecurity: predicting exploitability of vulnerabilities by description, Knowl. Based Syst. 210 (2020) 106529. https://doi.org/10.1016/j.knosys.2020.106529
[17] Y. Jia, Z. Gu, L. Du, Y. Long, Y. Wang, J. Li, Y. Zhang, Artificial intelligence enabled cyber security defense for smart cities: a novel attack detection framework based on the MDATA model, Knowl. Based Syst. 276 (2023) 110781. https://doi.org/10.1016/j.knosys.2023.110781
[18] S.L. Mirtaheri, R. Shahbazian, Machine learning: theory to Applications, CRC Press, 2022.
[19] Z. Han, X. Li, Z. Xing, H. Liu, Z. Feng, Learning to predict severity of software vulnerability using only vulnerability description, in: 2017 IEEE International conference on software maintenance and evolution (ICSME), IEEE, 2017, pp. 125–136.
[20] S.E. Sahin, A. Tosun, A conceptual replication on predicting the severity of software vulnerabilities, in: Proceedings of the 23rd International Conference on Evaluation and Assessment in Software Engineering, 2019, pp. 244–250.
[21] T.H.M. Le, B. Sabir, M.A. Babar, Automated software vulnerability assessment with concept drift, in: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR), IEEE, 2019, pp. 371–382.
[22] T.H.M. Le, H. Chen, M.A. Babar, A survey on data-driven software vulnerability assessment and prioritization, ACM Comput. Surv. 55 (5) (2022) 1–39.
[23] M. Bozorgi, L.K. Saul, S. Savage, G.M. Voelker, Beyond heuristics: learning to classify vulnerabilities and predict exploits, in: Proceedings of the 16th ACM SIGKDD International Conference on Knowledge discovery and Data Mining, 2010, pp. 105–114.
[24] C. Sabottke, O. Suciu, T. Dumitraş, Vulnerability disclosure in the age of social media: Exploiting twitter for predicting {Real-World} exploits, in: 24th USENIX Security Symposium (USENIX Security 15), 2015, pp. 1041–1056.
[25] M. Almukaynizi, E. Nunes, K. Dharaiya, M. Senguttuvan, J. Shakarian, P. Shakarian, Proactive identification of exploits in the wild through vulnerability mentions online, in: 2017 International Conference on Cyber Conflict (CyCon US), IEEE, 2017, pp. 82–88.
[26] M. Almukaynizi, E. Nunes, K. Dharaiya, M. Senguttuvan, J. Shakarian, P. Shakarian, Patch before exploited: an approach to identify targeted software vulnerabilities, AI Cybersecur. 151 (2019) 81–113.
[27] D.A. de Sousa, E.R. de Faria, R.S. Miani, Evaluating the performance of Twitter-based exploit detectors, arXiv:2011.03113. (2020).
[28] J. Jacobs, S. Romanosky, I. Adjerid, W. Baker, Improving vulnerability remediation through better exploit prediction, J. Cybersecur. 6 (1) (2020) tyaa015.
[29] O. Suciu, C. Nelson, Z. Lyu, T. Bao, T. Dumitraş, Expected exploitability: predicting the development of functional vulnerability exploits, in: 31st USENIX Security Symposium (USENIX Security 22), 2022, pp. 377–394.
[30] S. Ognawala, R.N. Amato, A. Pretschner, P. Kulkarni, Automatically assessing vulnerabilities discovered by compositional analysis, in: Proceedings of the 1st International Workshop on Machine Learning and Software Engineering in Symbiosis, 2018, pp. 16–25.
[31] M. Gawron, F. Cheng, C. Meinel, Automatic vulnerability classification using machine learning, in: Risks and Security of Internet and Systems: 12th International

Conference, CRiSIS 2017, Dinard, France, September 19–21, 2017, Revised Selected Papers 12, Springer, 2018, pp. 3–17.
[32] X. Gong, Z. Xing, X. Li, Z. Feng, Z. Han, Joint prediction of multiple vulnerability characteristics through multi-task learning, in: 2019 24th International Conference on Engineering of Complex Computer Systems (ICECCS), IEEE, 2019, pp. 31–40.
[33] K. Kanakogi, H. Washizaki, Y. Fukazawa, S. Ogata, T. Okubo, T. Kato, H. Kanuka, A. Hazeyama, N. Yoshioka, Tracing capec attack patterns from cve vulnerability information using natural language processing technique 12 (2021) 298.
[34] Y. Jiang, Y. Atif, An approach to discover and assess vulnerability severity automatically in cyber-physical systems, in: 13th International Conference on Security of Information and Networks, 2020, pp. 1–8.
[35] H. Guo, S. Chen, Z. Xing, X. Li, Y. Bai, J. Sun, Detecting and augmenting missing key aspects in vulnerability descriptions, ACM Trans. Softw. Eng. Methodol. (TOSEM) 31 (3) (2022) 1–27.
[36] A. Silva, S. Fang, M. Monperrus, RepairLLaMA: Efficient Representations and Fine-tuned Adapters for Program Repair, arXiv:2312.15698. (2023).
[37] J. Zhang, H. Wen, L. Deng, M. Xin, Z. Li, L. Li, H. Zhu, L. Sun, HackMentor: fine-tuning large language models for cybersecurity, in: 2023 IEEE 22nd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), IEEE, 2023, pp. 452–461.
[38] S.P. Mohammed, G. Hossain, ChatGPT in education, healthcare, and cybersecurity: opportunities and challenges, in: 2024 IEEE 14th Annual Computing and Communication Workshop and Conference (CCWC), IEEE, 2024, pp. 0316–0321.
[39] R.P. Khandpur, T. Ji, S. Jan, G. Wang, C.-T. Lu, N. Ramakrishnan, Crowdsourcing cybersecurity: cyber attack detection using social media, in: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, 2017, pp. 1049–1057.
[40] S. Zong, A. Ritter, G. Mueller, E. Wright, Analyzing the perceived severity of cybersecurity threats reported on social media, arXiv:1902.10680. (2019).
[41] X. Li, X. Ren, Y. Xue, Z. Xing, J. Sun, Prediction of vulnerability characteristics based on vulnerability description and prompt learning, in: 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2023, pp. 604–615. https://doi.org/10.1109/SANER56733.2023.00062
[42] G. Lu, X. Ju, X. Chen, S. Yang, L. Chen, H. Shen, Assessing the effectiveness of vulnerability detection via prompt tuning: an empirical study, in: 2023 30th Asia-Pacific Software Engineering Conference (APSEC), 2023, pp. 415–424. https://doi.org/10.1109/APSEC60848.2023.00052
[43] R. Malhotra, Vidushi, Text mining based an automatic model for software vulnerability severity prediction, Int. J. Syst. Assur. Eng. Manage. (2024) 1–19.
[44] Z. Ren, X. Ju, X. Chen, H. Shen, ProRLearn: boosting prompt tuning-based vulnerability detection by reinforcement learning, Autom. Softw. Eng. 31 (2) (2024) 38.
[45] S. Elder, M.R. Rahman, G. Fringer, K. Kapoor, L. Williams, A survey on software vulnerability exploitability assessment, ACM Comput. Surv. 56 (8) (2024). https://doi.org/10.1145/3648610
[46] D. Levshun, Comparative analysis of machine learning methods in vulnerability categories prediction based on configuration similarity, in: International Symposium on Intelligent and Distributed Computing, Springer, 2023, pp. 70–81.
[47] T. Lin, Y. Wang, X. Liu, X. Qiu, A survey of transformers, AI Open 3 (2022) 111–132.
[48] S.L. Mirtaheri, A. Pugliese, et al., Leveraging generative AI to enhance automated vulnerability scoring, in: IEEE Cyber Science and Technology Congress (CyberSciTech), 2024a, pp. 618–625.
[49] S.L. Mirtaheri, A. Pugliese, N. Movahed, A. Majd, et al., Advanced automated vulnerability scoring: improving performance with a fine-tuned BERT-CNN model, in: International Symposium on Telecommunication (IST), 2024b.
[50] S.L. Mirtaheri, S. Greco, R. Shahbazian, A self-attention TCN-based model for suicidal ideation detection from social media posts, Expert Syst. Appl. 255 (2024c) 124855. https://doi.org/10.1016/j.eswa.2024.124855
[51] S.L. Mirtaheri, A. Pugliese, N. Movahed, R. Shahbazian, A comparative analysis on using GPT and BERT for automated vulnerability scoring, Intell. Syst. Appl. 26 (2025) 200515.
[52] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, Adv. Neural Inf. Process. Syst. 30 (2017) 5998–6008.
[53] J. Vig, A multiscale visualization of attention in the transformer model, arXiv:1906.05714. (2019).
[54] M.T. Ribeiro, S. Singh, C. Guestrin, "Why should I trust you?" Explaining the predictions of any classifier, in: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, 2016, pp. 1135–1144.
[55] S.M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, Adv. Neural Inf. Process. Syst. 30 (2017) 4768–4777.
[56] D. Slack, S. Hilgard, E. Jia, S. Singh, H. Lakkaraju, Fooling lime and shap:adversarial attacks on post hoc explanation methods, in: Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society, 2020, pp. 180–186.
[57] A. Azizi, I.A. Tahmid, A. Waheed, N. Mangaokar, J. Pu, M. Javed, C.K. Reddy, B. Viswanath, {T-Miner}: a generative approach to defend against trojan attacks on {DNN-based} text classification, in: 30th USENIX Security Symposium (USENIX Security 21), 2021, pp. 2255–2272.
[58] J. Zhou, Y. Chen, C. Shen, Y. Zhang, Property inference attacks against GANs, arXiv:2111.07608. (2021).
[59] J. Zhang, Z. Chen, H. Zhang, C. Xiao, B. Li, {DiffSmooth}: certifiably robust learning via diffusion models and local smoothing, in: 32nd USENIX Security Symposium (USENIX Security 23), 2023, pp. 4787–4804.
[60] S. An, G. Tao, Q. Xu, Y. Liu, G. Shen, Y. Yao, J. Xu, X. Zhang, Mirror: model inversion for deep learning network with high fidelity, in: Proceedings of the 29th Network and Distributed System Security Symposium, 2022.
[61] Y. Xiao, Q. Yuan, K. Jiang, J. He, X. Jin, L. Zhang, EDiffSR: an efficient diffusion probabilistic model for remote sensing image super-resolution, IEEE Trans. Geosci. Remote Sens. 62 (2023) 1–14.